Course Project: Data Structures

(1st Semester 2023-2024)

SUBMISSION BY 14th WEEK

COURSE PROJECT INSTRUCTIONS

- You can work in a group, and there will be three students in each project group.
- You can choose any one of the projects from the mentioned projects. You can also choose project of your own choice **but its appropriateness must be confirmed by me**.
- **Students are required to:**
  1. Implement the proposed algorithm using the C++ programming language
  2. Prepare project report
  3. Demonstrate the completed project's solution

PROJECT CODE:

- The code should be written with meaningful functions and variable names.
- Add appropriate comments in the program.

PROJECT REPORT:

The final project report should contain the following items:

1. Cover Page
   - College and KFU logo
   - Project title
   - Student names and id
2. Table of contents
3. Introduction (Project Description)
4. Your methodology (solution) including
   - Algorithm /Pseudocode
   - Executable Program Code.
5. Results and discussion
   - Inputs and Outputs (Screenshots)
6. Conclusion

PROJECT DEMO PRESENTATION:

- The project demo presentation should reflect the report.
- Students should be able to explain any questions regarding the project.

GRADING POLICY:

Project Code	50 Marks
Project Report	30 Marks
Project demo presentation	20 Marks

Total **100 Marks**

PLAGIARISM WILL BE STRICTLY DEALT WITH RESULTING IN

- **OBTAINING ZERO MARK**
- **FACE THE ACTIONS FROM DEPT. COUNCIL**

1. MICRO-VERSION OF SOCIAL NETWORKING!!

In this project, you will have to implement the concept of a social networking website using **Graph data structure** by implementing them using an Adjacency List. Use **ID** as the unique identification of each user in your solution.

SAMPLE INPUTS:
Specifically, your program will have the following menu:

1. Signup a new account (username, password, address)
2. Login User (User can log in to their account using their name and password)
3. Search for a user
4. Friendship Request (You can send a request to another user)
5. Cancel a friend request
6. Accept a friend request
7. Show friends
8. Show suggested friends on the base of the address added during signup.
9. Show mutual friends
10. Show your post
11. Update a post related to something (text form)
0. Exit

SAMPLE OUTPUTS:

```
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice:
```

MENU

```
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 1
Enter name: User1
Enter password: 1234
Enter address: Hofuf
User signed up with ID: 1
```

1. SIGN UP

```
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 2
Enter name: User1
Enter password: 1234
Logged in as User1 (ID: 1)
Menu:
```

2. LOG IN

```
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 3
Enter name to search: User1
User found: User1, Address: Hofuf
```

3. SEARCH USER

```
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 4
Enter your ID: 1
Enter friend's ID: 2
Send Friend
```

4. SEND REQUEST

```
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 5
Enter your ID: 2
Enter friend's ID: 1
Cancel Friend Request
```

5. CANCEL REQUEST

```
Enter your choice: 4
Enter your ID: 1
Enter friend's ID: 3
Send Friend Request
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 6
Enter your ID: 3
Enter friend's ID: 1
Accept Friend Request
```

6. ACCEPT REQUEST

```
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 7
Enter your ID: 3
Friends of User3:
User1
```

7. SHOW FRIENDS

```
Enter your choice: 3
Enter name to search: User4
User found: User4, Address: Hofuf
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 3
Enter name to search: User1
User found: User1, Address: Hofuf
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 8
Enter your ID: 1
Suggested friends for User1:
User4
```

8. SHOW SUGGESTED FRIENDS

```
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 9
Enter your ID: 1
Enter friend's ID: 3
Mutual friends between User1 and User3:
User2
```

9. SHOW MUTUAL FRIENDS

```
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 10
Enter your ID: 1
Posts of User1:
Hi, this is data structure project
```

10. SHOW POST

```
Menu:
1. Signup
2. Login
3. Search User
4. Send Friend Request
5. Cancel Friend Request
6. Accept Friend Request
7. Show Friends
8. Show Suggested Friends
9. Show Mutual Friends
10. Show Posts
11. Update Post
0. Exit
Enter your choice: 11
Enter your ID: 1
Enter post: Hi. this is updated post
```

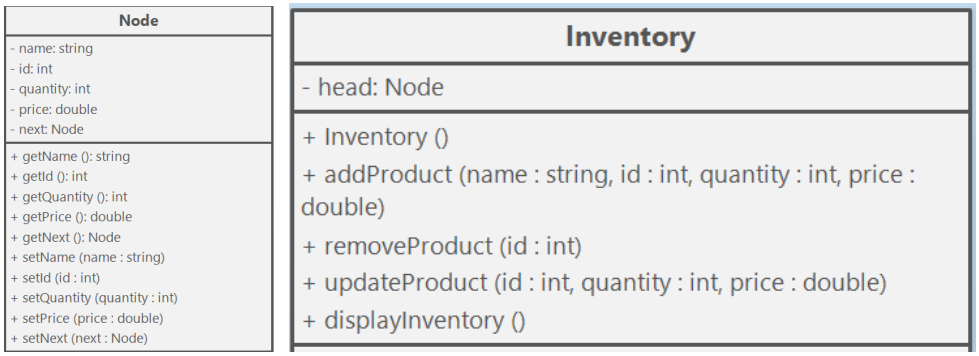
11. UPDATE POST

2. INVENTORY MANAGEMENT SYSTEM!!

Assume you're working as a computer programmer for a company that sells a variety of products. The company wishes to use a linked list data structure to keep track of its inventories. You've been given the job of developing a solution that would enable the company to use a linked list to add, delete, and update products in their inventory.

SYSTEM REQUIREMENTS:

- You are restricted to using just the **Linked List data structure** for this project. The usage of any other data structure will result in a grade reduction.
- Do not use struct. You have to implement the **Node class**. Using struct will result in the deduction of marks
- Each product in inventory is represented by one "node" in the Linked List data structure. Each node should contain the product name, product ID, quantity, and price.
- The following operations need to be completed by your program on the output console:
  - The user should be able to **add** new products to inventory by entering the product's name, product ID, quantity, and price.
  - The user should be able to **delete** a product from the inventory by providing the product ID.
  - The user should be able to **update** the quantity or selling price of a product by providing the product ID.
  - The user should be able to **display** all available inventory, along with product names, IDs, quantities, and prices.
- You should include the following classes in your solution:
  - Class node for archiving product data.
  - Inventory Type: Maintain Linked List of All Items



SAMPLE INPUTS:

Specifically, your program will have the following menu:

1. Add product
2. Remove product
3. Update product
4. Display inventory
5. Exit

SAMPLE OUTPUTS:

```
C:\Users\ROG\Desktop\ConsoleApplication2\Debug\Console...
Please choose an action:
1. Add product
2. Remove product
3. Update product
4. Display inventory
0. Exit
Enter your choice: 1
How many products do you want to add? 3
Enter product name: Biscuits
Enter product ID: 1
Enter product quantity: 30
Enter product price: 60
Product added successfully.
Enter product name: Colddrinks
Enter product ID: 2
Enter product quantity: 40
Enter product price: 120
Product added successfully.
Enter product name: Icecream
Enter product ID: 3
Enter product quantity: 10
Enter product price: 50
Product added successfully.
Please choose an action:
1. Add product
2. Remove product
3. Update product
4. Display inventory
0. Exit
Enter your choice: 2
Enter product ID to remove: 2
Product removed successfully.
Please choose an action:
1. Add product
2. Remove product
3. Update product
4. Display inventory
0. Exit
Enter your choice: 3
Enter product ID to update: 1
Enter new quantity: 15
Enter new price: 30
Product updated successfully.
Please choose an action:
1. Add product
2. Remove product
3. Update product
4. Display inventory
0. Exit
Enter your choice: 2
Enter product ID to remove: 2
Product removed successfully.
Please choose an action:
1. Add product
2. Remove product
3. Update product
4. Display inventory
0. Exit
Enter your choice: 3
Enter product ID to update: 1
Enter new quantity: 15
Enter new price: 30
Product updated successfully.
Please choose an action:
1. Add product
2. Remove product
3. Update product
4. Display inventory
0. Exit
Enter your choice: 4
Inventory:
Name     ID      Quantity        Price
Biscuits         1       15              30
Icecream         3       10              50
```

3. Where is the Marble?

Raju and Meena love to play with Marbles. They have a lot of marbles with numbers written on them. In the beginning, Raju would place the marbles one after another in ascending order of the numbers written on them. Then, Meena would ask Raju to find the first marble with a certain number. She would count 1...2...3. Raju gets one point for the correct answer, and Meena gets the point if Raju fails. After some fixed number of attempts, the game ends and the player with maximum points wins. Today it's your chance to play as Raju. Being a smart kid, you have in your advantage the computer. But don't underestimate Meena, she wrote a program to keep track of how much time you're taking to give all the answers. So now you have to write a program, which will help you in your role as Raju.

Input

There can be multiple test cases. The total number of test cases is less than 65. Each test case consists begins with 2 integers: **N** the number of marbles and **Q** the number of queries Meena would make. The next **N** lines would contain the numbers written on the **N** marbles. These marble numbers will not come in any particular order. Following **Q** lines will have **Q** queries. Be assured, none of the input numbers are greater than 10000 and none of them are negative.

Input is terminated by a test case where **N** = 0 and **Q** = 0.

Output

For each test case output there must be a serial number of the test case. For each query, write one line of output. The format of this line will depend on whether the number is consulted or not written in one of the marbles.

The two different formats are described below:
'x found at y', if the first marble with number x was found at position y. Positions are numbered *1, 2,..., N*.
'x not found', if the marble with number x is not present.

Input Sample	Output Sample
4 1 2 3 5 1 5 5 2	CASE# 1: 5 found at 4 CASE# 2: 2 not found 3 found at 3

Input Sample	Output Sample
1 3 3 3 1 2 3 0 0	

4. Regular Bracket Sequences

A bracket sequence is a string containing only characters "(" and ")". A regular bracket sequence is a bracket sequence that can be transformed into a correct arithmetic expression by inserting characters "1" and "+" between the original characters of the sequence. For example, bracket sequences "()()" and "(())" are regular (the resulting expressions are: "(1)+(1)" and "((1+1)+1)"), and ")(", "(" and ")" are not.

You are given an integer nn. Your goal is to construct and print **exactly** nn different regular bracket sequences of length 2n2n.

Input

The first line contains one integer tt (1≤t≤501≤t≤50) — the number of test cases.

Each test case consists of one line containing one integer nn (1≤n≤501≤n≤50).

Output

For each test case, print nn lines, each containing a regular bracket sequence of length **exactly** 2n2n. All bracket sequences you output for a testcase should be different (though they may repeat in different test cases). If there are multiple answers, print any of them. It can be shown that it's always possible.

Example

input

Copy

```
3
3
1
3
```

output

Copy

```
()()()
((()))
(()())
()
```

```
((()))
(())()
()(())
```

5. SAM's First Dictionary

Sam, 8, has a dream - he wants to produce his very own dictionary. This is not an easy task for him, as the number of words that he knows is, well, not quite enough. Instead of thinking up all the words himself, he has a briliant idea. From his bookshelf he would pick one of his favourite story books, from which he would copy out all the distinct words. By arranging the words in alphabetical order, he is done! Of course, it is a really time-consuming job, and this is where a computer program is helpful.

You are asked to write a program that lists all the different words in the input text. In this problem, a word is defined as a consecutive sequence of alphabets, in upper and/or lower case. Words with only one letter are also to be considered. Furthermore, your program must be CaSe InSeNsItIvE. For example, words like "Apple", "apple" or "APPLE" must be considered the same.

Input

The input file is a text with no more than 10000 lines. An input line has at most 200 characters. Input is terminated by EOF.

Output

Your output should give a list of different words that appears in the input text, one in a line. The words should all be in lower case, sorted in alphabetical order. You can be sure that he number of distinct words in the text does not exceed 5000.

Sample Input	Sample Output
Ex(*$a#.mpl.e: Adventures in Disneyland Two blondes were going to Disneyland when they came to a fork in the road. The sign read: "Disneyland LEFT." So they went home.	a adventures blondes came disneyland e ex fork going home in left mpl

Sample Input	Sample Output
	read road sign so the they to two went were when

6. Economic Phonebook

Due to the large number of complaints, the telephone company of St. Petersburg is being forced to invest heavily in improving its services. To do this, the company decided to reduce the budget of some sectors to increase other more essential. One of the sectors that will have its budget reduced is the printing of phone books.

With a reduced budget, the sector of printing phonebooks cannot buy enough toner to print the complete lists. As telephone numbers are printed vertically aligned, it was suggested the following solution: from the second phone number printed, the initial digits of the next number to be printed that match the above number are omitted, leaving only one space white. For example, for the numbers 535456, 535488, 536566 and 835456 would be printed is as follows:

```
7.  5 3 5 4 5 6
          8 8
      6 5 6 6
    8 3 5 4 5 6
```

Note that this impression has saved the impression 6 characters. The telephone company also wondered not print the repeated suffixes, but in tests saw that the answer was not good for the user and therefore decided to make only the elimination prefixes. To determine whether the economy will be sufficient, the printing industry wants to know the maximum number of characters that can be omitted. However, as in any big city, there are several phone numbers and they do not want to spend man-hours to manually calculate this value. So it's up to you, new employee of the company, automate the economy made by toner, the number of characters.

Input

The input consists of various test cases and ends with the end of file (EOF). Each test case contains an integer **N**, which reports the number of phones in the list. The next **N** ($1 \leq \mathbf{N} \leq 10^5$) lines have, each one, a phone **X***i*, up to 200 characters. For one test case phone numbers have the same number of characters. A phone number can start with the character '0 '.

Output

For each test case print one line indicating the maximum number of characters saved by this process.

Sample Input	Sample Output
2 12345 12354 3 535456 535488 835456	3 4